STUDENT

S-0004-EPJ

TENTAMEN

240822 DT141G, DT201G, DT202G Sundsvall

Kurskod	--
Bedömningsform	--
Starttid	22.08.2024 08:00
Sluttid	22.08.2024 13:00
Bedömningsfrist	--
PDF skapad	05.11.2025 13:12
Skapad av	Liselott Engelbrekts

i

Mittuniversitetet
MID SWEDEN UNIVERSITY

Tentamen

DT141G Operativsystemsteori
DT201G, DT202G Operativsystem

Jimmy Åhlander

Instruktioner

Tid	5 timmar
Resurser	Kalkylator, kurslitteratur, lexikon, egna anteckningar, penna och sudd.
Poäng	100
Krav för godkänt	För godkänt betyg på tentamen fordras betyg E eller högre.
Preliminära betygsgränser	A 90 % B 80 % C 70 % D 60 % E 50 %
Allmänt	Använd stycken, punktlistor och annan textformatering där det är lämpligt när du besvarar frågor med text. Om frågan kräver att du motiverar ditt svar, visar dina beräkningar eller ritar en figur erhålles i normalfallet inga poäng för endast svaret. Ibland är det dock endast en rekommendation och inte ett krav att svara på ett visst sätt. Exempel på krav: "*Rita en figur som styrker ditt svar*", "*Svara i bytes*", "*Visa dina beräkningar*". Exempel på rekommendationer: "*Rita en figur om du anser att det styrker ditt svar*", "*Tips: Skapa en tabell*". Tentan är anonymiserad. Skriv aldrig ditt namn, personnummer eller annan identifierande information i dina svar. Det är inte tillåtet att använda eller ha tillgång till mobiltelefon, surfplatta eller liknande enheter under tentamen. Vid tekniska problem, kontakta alltid tentamensvakt. Stäng inte av eller starta om datorn utan uppsikt, om du inte redan lämnat in tentamen.

InsperaScan används ej för denna tentamen.

Mittuniversitetet
Institutionen för data- och elektroteknik (DET)

Jimmy Åhlander, 010 - 142 86 90

ℹ Filer hanteras på olika sätt i olika operativsystem. Generellt gäller emellertid att de har attribut och rättigheter samt att de lagras i någon form av katalogstruktur.

1(a) I både Windows- och Linuxmiljöer används olika prefix för filstorlekar, om än med olika betydelse. Anta att NTFS respektive ext4 används som filsystem med Explorer respektive Nautilus som grafisk filhanterare i ett Windows- respektive Linuxsystem.

Givet en fil av storleken 8 000 bytes. Hur kommer **storleken** presenteras i nedan angivna filhanterare för de olika enheterna/prefixen? *Avrunda till två decimalers precision.*

Explorer: 8000 bytes.

Explorer: 7.81 KB.

Nautilus: 8000 bytes.

Nautilus: 8 kB.

Givet en blockstorlek på 4 096 bytes, hur kommer **storleken på disk** presenteras för filen i nedan angiven filhanterare? *Avrunda till två decimalers precision.*

Explorer: 15 bytes.

Explorer: 0.01 KB.

Delvis rätt. 4 av 6 poäng.

1(b) I följande bild ses delar av innehållet i katalogen där bland annat **`file.txt`** återfinnes med hjälp av kommandot **`ls -lia`** i **`Bash`**. Besvara följande frågor utifrån denna information.

```
jimahl@ubuntu:~/test/temp$ ls -lia
total 12
12620 drwxrwxr-x  2 jimahl jimahl 4096 Aug 15 07:13 .
 2314 drwxrwxr-x 12 jimahl jimahl 4096 Aug 15 07:15 ..
12623 -rw-r--rwx  2 jimahl jimahl   15 Aug 15 07:13 file.txt
```

Hur skulle filrättigheterna för file.txt uttryckas på numeriskt (oktalt) format?

Svar: 647 .

Vilken grupp tillhör file.txt?

Svar: jimahl .

Vilket indexnummer (inode) har file.txt?

Svar: 12623 .

Hur många hårda länkar leder till samma inode som file.txt (*från vad du kan se*)?

Svar: 2 stycken.

Hur många symboliska (mjuka) länkar leder till file.txt (*från vad du kan se*)?

Svar: 0 stycken.

Rätt. 5 av 5 poäng.

1(c) Utgå ifrån tidigare presenterad information för att även besvara följande: Vilka av nedan angivna operationer är möjliga för användaren **`jimahl`**?

Välj ett eller flera alternativ:

- [x] Ändra filinnehållet i **`file.txt`**.
- [ ] Exekvera **`file.txt`**, t.ex. som ett skalskript.
- [x] Skriva ut innehållet i **`file.txt`**, t.ex. med **`cat`**.
- [x] Ändra namnet på **`file.txt`**, t.ex. med **`mv`**.
- [x] Skriva ut innehållet i **`~/test/temp`**, t.ex. med **`ls`**.
- [x] Skapa nya filer i **`~/test/temp`**, t.ex. med **`touch`**.
- [x] Ta bort filer i **`~/temp/test`**, t.ex. med **`rm`**.
- [ ] Ta bort katalogen **`~/test/temp`**, t.ex. med **`rm -r`**.

Tips:

- Filnamn inklusive filändelser säger inget om filformatet.

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Delvis rätt. 4.5 av 5 poäng.

i Sidersättningsalgoritmer tillämpas i operativsystem som nyttjar virtuellt minne och demand paging. Givet tre fysiska ramar och sex logiska sidor, anta att processen P_1 använder sina logiska sidor i en sekvens angiven av en referenssträng där R representerar en läsoperation och W representerar en skrivoperation.

Processen P_1 har redan tre sidor allokerade i det fysiska minnet, se Tabell 1. P_1 behöver nu sidor enligt följande referenssträng: **`W3 W4 W2 R2 W3 R1 R4 R3 R4`**.

Tabell 1. Sidtabell för P_1.

Sida	Ram	SC bit	Smutsig/mod bit	Valid bit
0				0
1	0	1	0	1
2	1	0	1	1
3	2	0	0	1
4				0
5				0

Bilderna nedan representerar primärminnet uppdelat i sina ramar vid 10 olika tidpunkter: en för utgångsläget plus en för varje kommande sidreferens. Pekaren som leder till huvudet i den cirkulära offerkön representeras av symbolen **>**.

2(a) Visa hur sidorna i referenssträngen kommer att allokeras till olika ramar om algoritmen **Second Chance (SC)** tillämpas genom att fylla i vilken sida som är allokerad till vilken ram vid varje tidpunkt, det vill säga efter varje referens i referenssträngen.

Endast sidnumret ska anges i respektive fält.

>1	1	1	2	2	2	2	4	4	4
2	2	4	4	4	4	1	1	1	1
3	3	3	3	3	3	3	3	3	3

Tips:

- Skissa först med penna och papper.

Delvis rätt. 3.15 av 5 poäng.

2(b) Hur många sidfel uppstår när SC tillämpas i det givna scenariot?

Antal sidfel: 4 .

Fel. 0 av 1 poäng.

2(c) Visa hur sidorna i referenssträngen kommer att allokeras till olika ramar om algoritmen **Optimal Page Replacement (OPT)** tillämpas genom att fylla i vilken sida som är allokerad till vilken ram vid varje tidpunkt, det vill säga efter varje referens i referenssträngen.

Endast sidnumret ska anges i respektive fält.

>1	1	4	4	4	4	4	4	4	4
2	2	2	2	2	2	1	1	1	1
3	3	3	3	3	3	3	3	3	3

Tips:

- Beakta sidtabellen ovan.

Delvis rätt. 0.9 av 5 poäng.

2(d) Hur många sidfel uppstår när OPT tillämpas i det givna scenariot?

Antal sidfel: 2 .

Rätt. 1 av 1 poäng.

ℹ Anta att du har ett exotiskt 10-bit operativsystem där kopplingen mellan logiskt och fysiskt minne sker med ett simpelt relocation register i MMU:n. Med andra ord använder systemet sammanhängande allokering och inte paging. Åtkomsttiden till primärminnet har i detta system uppmätts till 50 ns.

3(a) Hur mycket logiskt minne kan en process maximalt använda i detta system? *Svara i KiB. Ignorera eventuella implementationsspecifika begränsningar.*

Svar: 1 KiB.

Rätt. 2 av 2 poäng.

3(b) Vi byter nu från sammanhängande allokering till paging i systemet, med sidstorleken 128 B. Detta får ses som främst en teoretisk exercis då paging knappast är användbart i ett 10-bitars system. I vilket fall används ingen translation lookaside buffer (TLB).

Paging som teknik lider generellt av intern fragmentering. Hur mycket minne kan en process maximalt förlora till intern fragmentering i det här systemet? *Anta att processen nyttjar allokerade sidor i så stor utsträckning som möjligt. Svara i bytes.*

Svar: 127 B.

Rätt. 2 av 2 poäng.

3(c) Hur många sidor består nu det logiska minnet av?

Svar: 8 sidor.

Rätt. 2 av 2 poäng.

3(d) Processorn i systemet har precis genererat adressen 500_{10}. Vad blir sidnumret och förskjutningen för adressen?

Sidnummer: 3 .

Förskjutning: 116 .

Tips:

- Du kan svara i valfri talbas av följande: 2, 10, 16 utifrån den du tycker är enklast att arbeta med. Ange dock *inte* talbasen i fälten.

Rätt. 4 av 4 poäng.

3(e) Den effektiva åtkomsttiden till minnet förändras med en konstant faktor x när paging används istället för sammanhängande allokering. Under antagandet att ingen translation lookaside buffer (TLB) används, vad är faktorn x?

Välj ett alternativ

- 0
- 1
- 2
- 3
- 5
- 10

Tips: En faktor $x = 1$ innebär att den effektiva åtkomsttiden är oförändrad.

Rätt. 3 av 3 poäng.

3(f)

Vi vill nu minimera eventuella tidsförluster genom att också introducera en TLB för minneshanteringen. Den effektiva åtkomsttiden med paging och buffert får sammanlagt **inte öka med *mer än* 15 %** jämfört med den ursprungliga åtkomsttiden till primärminnet.

Vi har fått fem olika bufferts att välja mellan för att uppfylla kravet. Snabbare och träffsäkrare TLBs kostar mer varför vi önskar välja en så billig buffert som möjligt som samtidigt uppfyller vårt krav.

Vilken är den billigaste bufferten som uppfyller kravet?

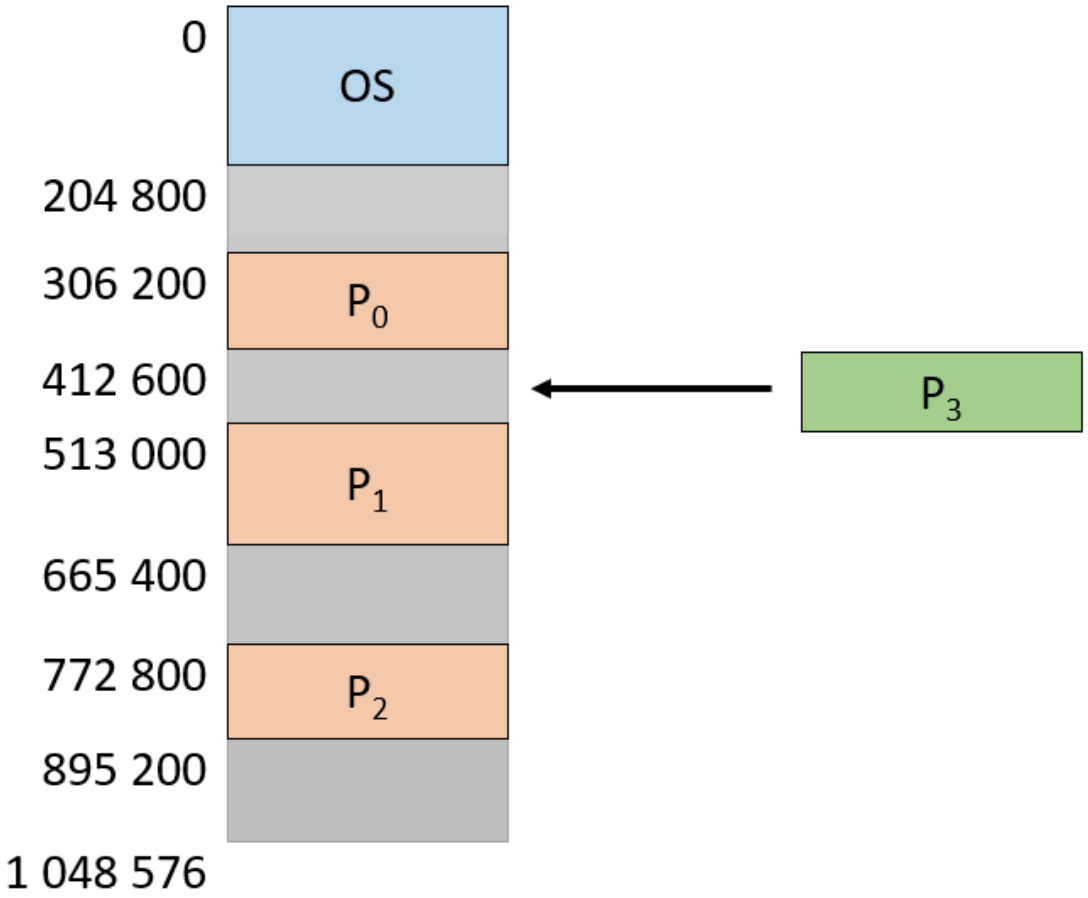
Välj ett alternativ:

- **`billigast:`**
  $85\ \%$ träffchans, $12\ ns$ åtkomsttid
- **`billigare:`**
  $87\ \%$ träffchans, $12\ ns$ åtkomsttid
- **`medeldyr:`**
  $85\ \%$ träffchans, $10\ ns$ åtkomsttid
- **`dyrare:`**
  $90\ \%$ träffchans, $8\ ns$ åtkomsttid
- **`dyrast:`**
  $95\ \%$ träffchans, $5\ ns$ åtkomsttid
- Ingen buffert uppfyller kravet.

Rätt. 5 av 5 poäng.

ℹ Sammanhängande allokering används för att placera processer i sammanhängande minnesspann. I följande scenario behöver utrymme allokeras för en ny process P_3 som kräver 100 KiB minne. I figuren anges den adress varje minnesspann börjar på. Operativsystemet är exempelvis placerat i spannet från och med adress 0 till och med adress 204799. Observera att figuren inte är helt skalenlig.

Besvara följande frågor relaterade till det givna scenariot.



4(a) På vilken adress kommer processens bas placeras om metoden **first-fit** används? *Svara med talbas 10*.

Ange adressen här: 665400 .

Rätt. 2 av 2 poäng.

4(b) På vilken adress kommer processens bas placeras om metoden **best-fit** används? *Svara med talbas 10*.

Ange adressen här: 665400 .

Rätt. 2 av 2 poäng.

4(c) På vilken adress kommer processens bas placeras om metoden **worst-fit** används? *Svara med talbas 10*.

Ange adressen här: 895200 .

Rätt. 2 av 2 poäng.

4(d) Om vi fortsätter att skapa fler processer, så långt det går, och samtidigt antar att alla processer som skapas är av samma storlek som P_3, hur mycket minne kommer vi då till sist sammanlagt ha förlorat till extern fragmentering och intern fragmentering? Du kan anta att inga processer någonsin stängs av. *Svara i bytes*.

Extern fragmentering: 201800 B.

Intern fragmentering: 201800 B.

Fel. 0 av 4 poäng.

i Anta att du har ett program vars uppgift är att iterativt inkrementera värdet ett hundra gånger i en global variabel vars initiala värde är noll, likt följande kodexempel:

```
1 x = 0
2 for i in range(100):
3     x += 1
```

Istället för att addera värdena iterativt i en tråd har arbetet istället fördelats över 2 olika trådar där vardera tråd fått i uppgift att utföra 50 inkrementeringar på den globala variabeln. Detta med avsikten att fördela arbetet över två processorkärnor för att öka exekveringshastigheten. Vid exekveringen förväntar du dig ett deterministiskt utfall men får istället, förvirrande nog, olika utdata efter varje körning.

5(a) Vilka av följande värden, om något alls och oavsett sannolikhet, kan den globala variabeln erhålla efter en körning av det problematiska programmet?

Välj ett eller flera alternativ:

- [ ] -4
- [ ] 0
- [x] 4
- [x] 50
- [x] 98
- [x] 99
- [x] 100
- [ ] 101
- [ ] 200

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Rätt. 5 av 5 poäng.

5(b) Efter några justeringar till programkoden, där du bland annat implementerat mutex-lås för att skydda den kritiska sektionen så upptäcker du att utfallet nu blir deterministiskt med samma utdata efter varje körning. Du ändrar nu även datorsystemet till ett med fyra processorkärnor istället för två.

Vilka av följande värden, oavsett sannolikhet, kan den globala variabeln nu erhålla efter en körning av det justerade programmet?

Välj ett eller flera alternativ:

- [ ] -4
- [ ] 0
- [ ] 4
- [ ] 50
- [ ] 98
- [ ] 99
- [x] 100
- [ ] 101
- [ ] 200

Observera:

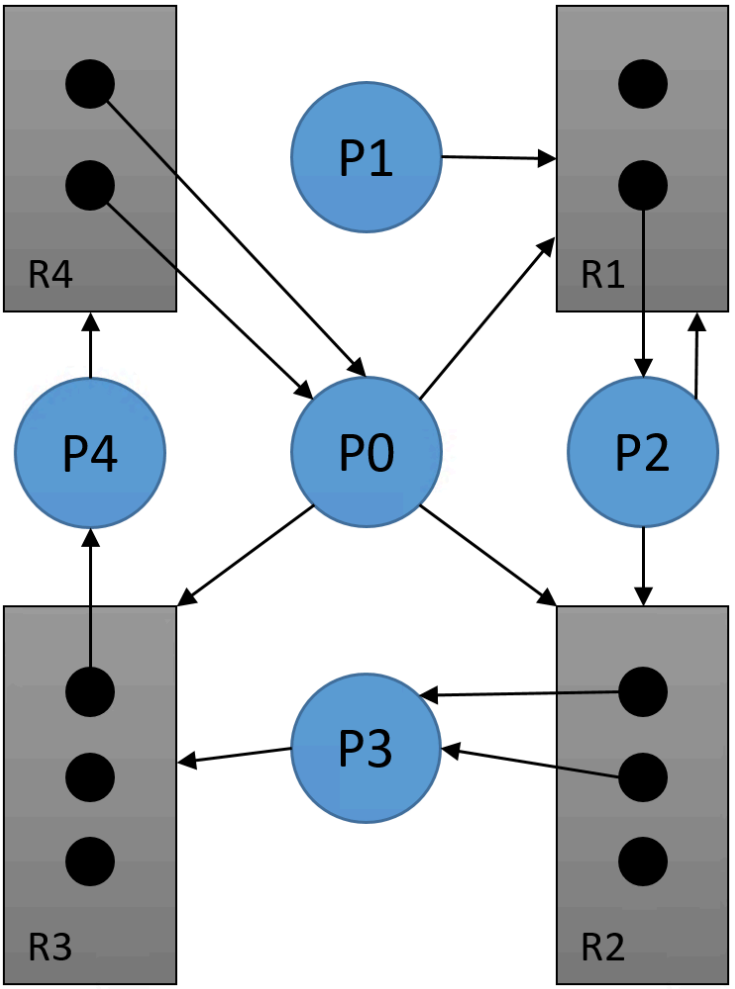
- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Rätt. 2 av 2 poäng.

6 Deadlocks kan uppstå när vissa kriterier är uppfyllda. I ett visst datorsystem råder scenariot som avbildas i resursallokeringsgrafen nedan. Observera att:

- Systemet har ingen preemption eller deadlockförhindrande algoritm.
- I de situationer där flera processer efterfrågar samma resurs samtidigt allokeras resurserna slumpartat.
- Processerna kan inte efterfråga fler resurser än de som redan representeras i grafen.
- Samtliga resursinstanser är odelbara.

Besvara följande frågor för det i grafen givna scenariot.



6(a) Har en deadlock uppstått? *Motivera ditt svar*.

Skriv in ditt svar här

P0 har potential att försätta systemet i en deadlock då den innehar bägge instanser av R4.

Om P0 tar den lediga instansen R1 och P2 tar den lediga instansen av R2 riskerar vi i det här läget att sätta vårt system i en deadlock. Detta då P0 inte kommer släppa sina resurs av R1 förrän den även erhållit en instans av R2, som i stunden ägs av P2.
P2 i sin tur kommer inte släppa sin instans av R2 förrän den erhålit en instans av R1. Så det har bildats en cykel mellan dessa processer som sätter systemet i ett unsafe state.

P0->P2->P0 och P2->P0->P2.

Ord: 105

Besvarad.

6(b) För att öka systemprestandan skulle vi vilja starta fler processer likt P_1. Hur många fler sådana processer, d.v.s. *utöver* den som redan startats, kan vi starta utan att försätta systemet i ett unsafe state där en deadlock skulle kunna uppstå?

Välj ett alternativ

- 0
- 1
- 2
- 3
- 4
- 5+. Det finns ingen övre gräns.

Det spelar ingen roll hur många fler processer likt P_1 som startas. Systemet är redan i ett unsafe state och riskerar att försättas i en deadlock.

Fel. 0 av 4 poäng.

CPU-schemaläggning används för att bestämma vilken process, eller när stöd finns, vilken tråd som ska exekvera på processorn. Trådar ankommer kontinuerligt till en redokö efter ett event eller avslutad I/O-operation.

I följande scenario, vid tidpunkten t har processorn precis blivit tillgänglig. En tråd har också precis ankommit till redokön i väntan på att bli schemalagd. Med ytterst avancerat estimerande (en spåkula) kan vi även se att fler trådar snart ankommer till kön, enligt nedan angivna ankomsttider.

Tabell. Redokön vid tidpunkten t.

Köplats (#)	Tråd	Uppskattad CPU-sprint (ms)	Ankomsttid (ms)
1	T_1	4	t
--	T_2	4	$t+1$
--	T_3	4	$t+2$
--	T_4	4	$t+3$

Du kan för dessa uppgifter anta att tidskostnaden för context switching är försumbar. Det är enklast att visualisera sig scenariot om du antar att $t = 0$ i starten.

7(a) Anta att trådarna schemaläggs med *Round Robin (RR)* där tidskvantumet är $q = 2\ ms$. Vad blir väntetiden för varje tråd och medelväntetiden (tiden varje tråd väntar i snitt)?

T1: 6 ms.

T2: 9 ms.

T3: 8 ms.

T4: 9 ms.

Medelväntetid: 8 ms.

Delvis rätt. 2 av 5 poäng.

7(b) Hur många context switches sker under förloppet ovan? Observera att ingen process är schemalagd på processorn vid scenariots start.

Svar: 8 context switches.

Rätt. 2 av 2 poäng.

7(c) Anta att du har möjlighet att ändra tidskvantumet i det tidigare scenariot. Vad är det *lägsta* tidskvantumet q som ger den *minsta möjliga* medelväntetiden?

q: 4 ms.

Rätt. 4 av 4 poäng.

ℹ I ett av dina serverkluster har du enligt loggad statistik de senaste dagarna haft:

- Ett CPU-nyttjande på i snitt 49 %.
- En överföringsbelastning på hårddiskarna där swap finns på 6 %.
- Ett I/O-nyttjande av övriga enheter på i snitt 4 %.

Se även följande information från top:

```
top - 15:16:44 up 98 days, 13:52, 1 user, load average: 0.10, 0.48, 0.48
Tasks: 222 total, 1 running, 221 sleeping, 0 stopped, 7 zombie
```

Då servern är en del av ett beräkningskluster är detta icke-önskvärd statistik. I idealfallet ska CPU-nyttjandet snitta minst 85 %, dag för dag.

8(a) Vilken eller vilka av följande åtgärder kan du tillämpa för att lösa eller delvis korrigera problemet?

Välj ett eller flera alternativ:

- [x] Installera en hårddisk med större lagringskapacitet.
- [ ] Installera en hårddisk med lägre lagringskapacitet.
- [x] Installera en hårddisk med högre överföringshastighet.
- [ ] Installera en hårddisk med lägre överföringshastighet.
- [ ] Installera en snabbare CPU.
- [x] Installera mer primärminne.
- [ ] Eliminera alla processer i zombietillstånd.
- [ ] Eliminera init/systemd.
- [ ] Öka graden av multiprogramming.
- [ ] Sänka graden av multiprogramming.
- [ ] Töm swaputrymmet.

Observera:

- Åtgärder som löser problemet är värda flera poäng.
- Åtgärder som bidrar till att delvis korrigera problemet är värda noll till enstaka poäng.
- Åtgärder som är inkorrekta är värda minuspoäng.
- Minimum är noll poäng.

Fel. 0 av 4 poäng.

9(a) Vad blir den maximala filstorleken och overheaden för en fil om länkad allokering används i ett system med 32 bitars adresser och en blockstorlek på 1024 B. *Svaren anges i GiB*.

Maximal filstorlek: 4096 GiB.

Overhead: 512 GiB.

Delvis rätt. 2 av 4 poäng.

9(b) Vad blir den maximala filstorleken och overheaden för samma fil om hierarkisk indexerad allokering används i ett system liknande UFS med samma adresstorlek och blockstorlek som i föregående uppgift. Anta att det finns 9 direkta, 1 indirekt och 1 dubbelt indirekt pekare. *Svaren anges i KiB respektive bytes.*

Maximal filstorlek: 65801 KiB.

Overhead: 264232 B.

Delvis rätt. 2 av 4 poäng.

9(c)

Anta att en trippelt indirekt pekare läggs till det tidigare nämnda UFS-liknande systemet. Hur mycket kommer nu den maximala filstorleken och overheaden att **öka med**? *Svaren anges i GiB respektive MiB.*

Ökning maximal filstorlek: 16 GiB.

Ökning overhead: 65 MiB. *Avrunda till närmsta heltal*.

Delvis rätt. 2 av 4 poäng.